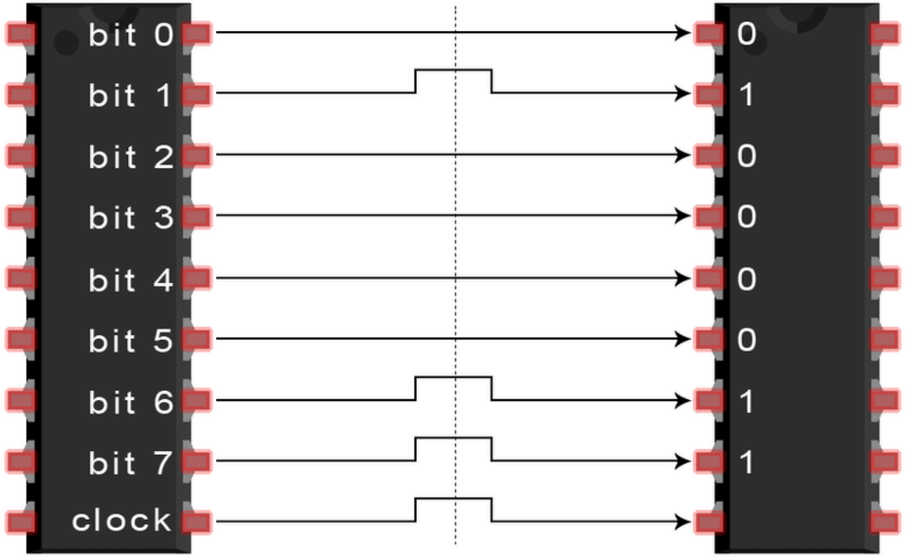
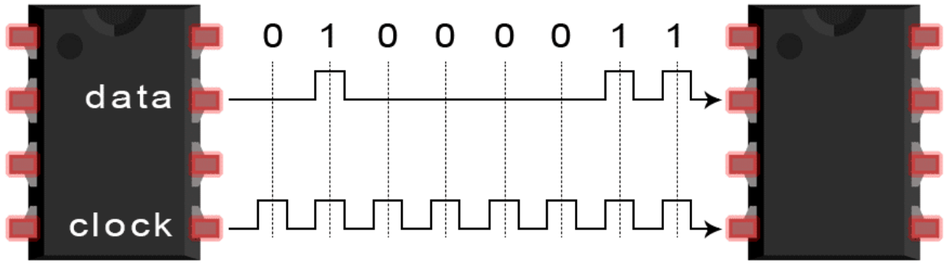
I2C, SPI and WI-FI. A deep dive into microntroller connectivity

Serial vs. Parallel Communication

Electronic devices talk to each other by sending bits of data through wires physically connected between devices





Terms:

Baud rate is a measure of the speed of data transfer, expressed in bits per second (bps).

Synchronous and asynchronous:
The primary difference between synchronous and asynchronous communication lies in how data is transmitted in relation to a clock signal, and whether or not both the sender and receiver need to be synchronized in time.

Synchronous communication relies on a shared clock signal to coordinate data transfer. The clock signal is typically generated by the master device and used by both the sender and receiver to ensure data is read or written at the same time.

Asynchronous communication does not use a shared clock signal. Instead, each data transmission includes special bits, such as start and stop bits, to indicate the beginning and end of data packets, allowing the receiver to determine the timing of each byte.

SPI: Serial Peripheral Interface

Overview:
SPI is a common communication protocol used by many different devices. For example, SD card reader modules, RFID card reader modules, and 2.4 GHz wireless transmitter/receivers all use SPI to communicate with microcontrollers.

MOSI (Master Output/Slave Input) – Line for the master to send data to the slave.
MISO (Master Input/Slave Output) – Line for the slave to send data to the master.
SCLK (Clock) – Line for the clock signal.
SS/CS (Slave Select/Chip Select) – Line for the master to select which slave to send data to.

How it works:

The clock signal:
It synchronizes the output of data bits from the master to the sampling of bits by the slave.
Any communication protocol where devices share a clock signal is known as **synchronous**

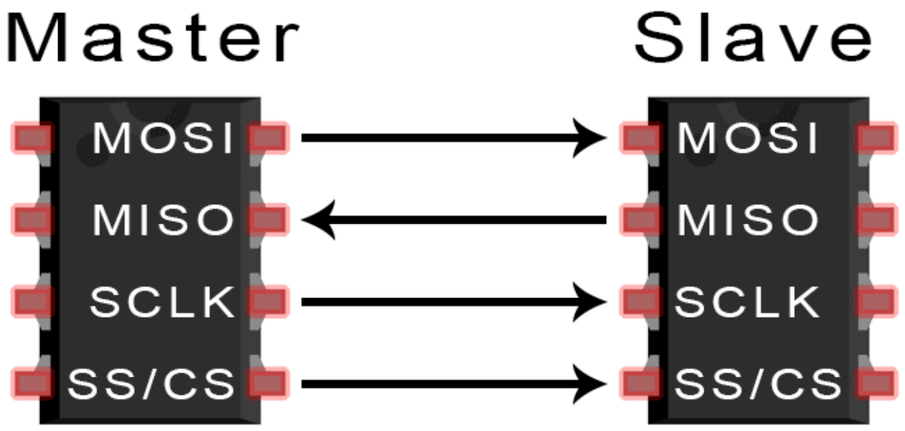
Slave Select:
The master can choose which slave it wants to talk to by setting the slave’s CS/SS line to a low voltage level.

MISO: Master In Slave Out
MOSI: Master Out Slave In

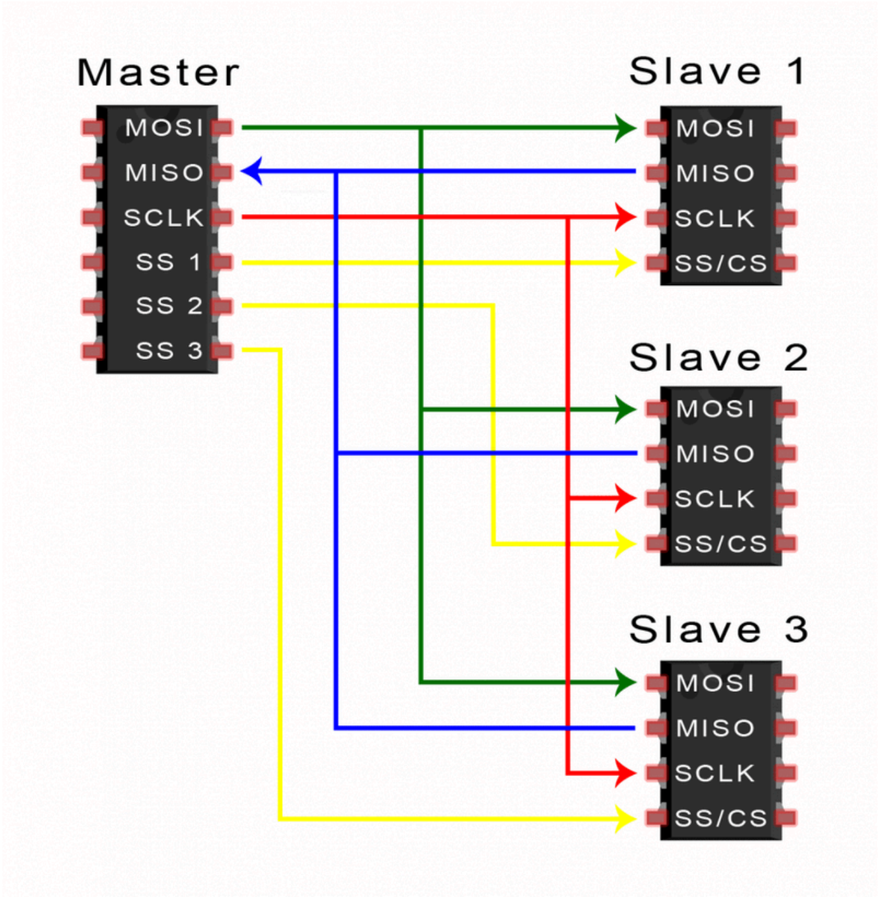
MOSI and MISO:

The master sends data to the slave bit by bit, in serial through the MOSI line. The slave receives the data sent from the master at the MOSI pin. Data sent from the master to the slave is usually sent with the most significant bit first.

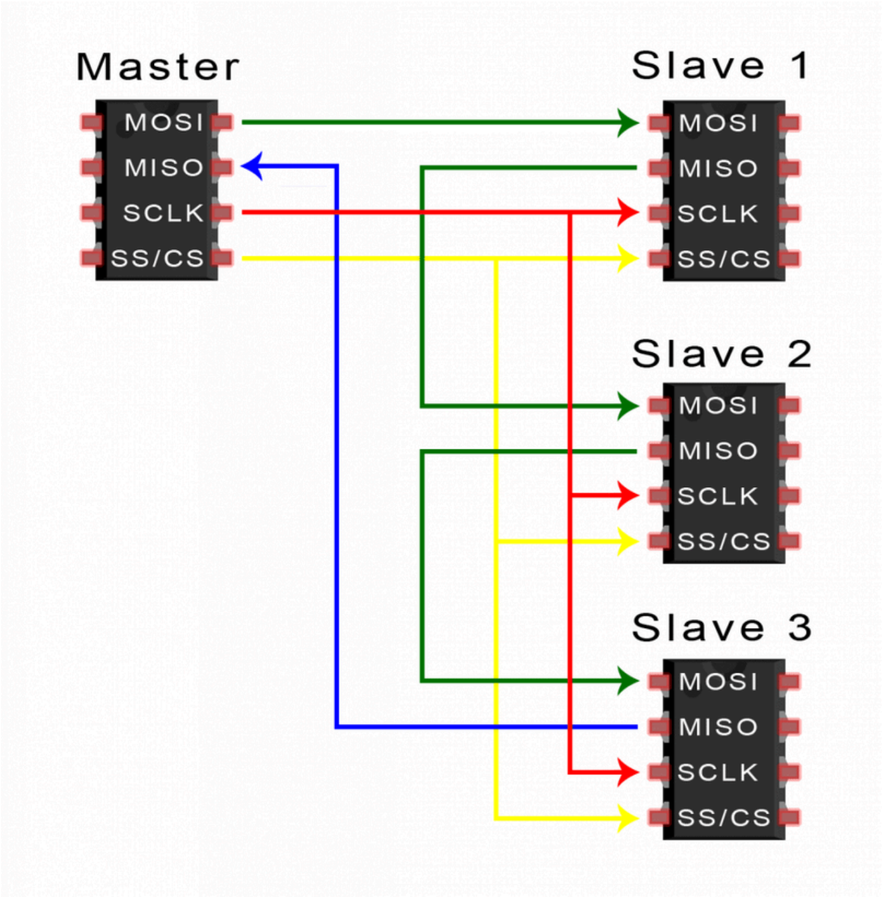
The slave can also send data back to the master through the MISO line in serial. The data sent from the slave back to the master is usually sent with the least significant bit first.



Multiple Slaves:



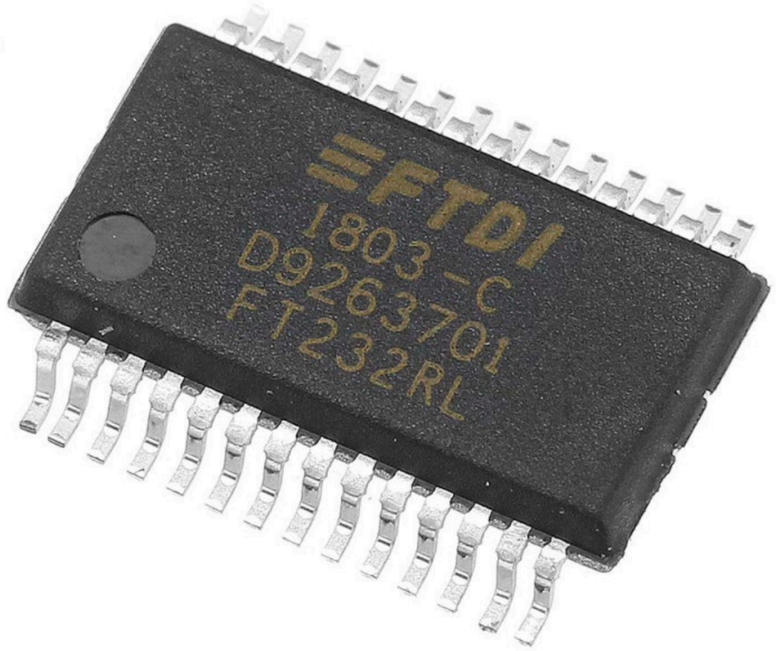
Chip select connected in parallel.



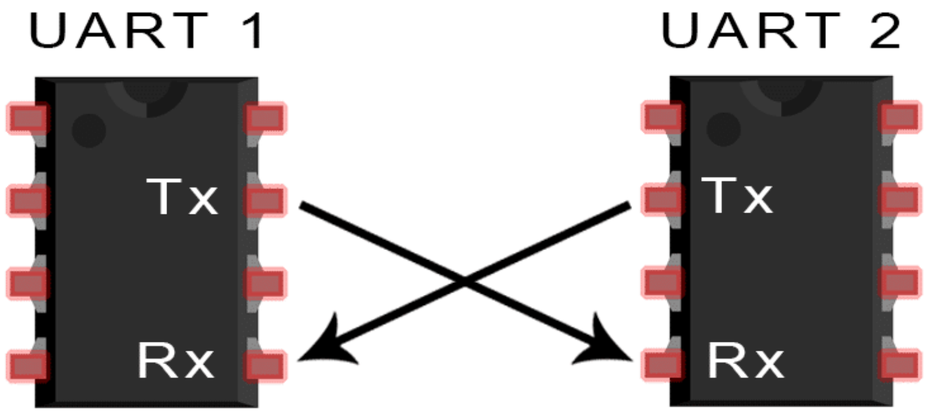
Chip select daisy-chained.

UART: Universal asynchronous Receiver Transmitter

It’s not a communication protocol like SPI and I2C, but a physical circuit in a microcontroller, or a stand-alone IC. A UART’s main purpose is to transmit and receive serial data.



Instead of a clock signal, the transmitting UART adds start and stop bits to the data packet being transferred.

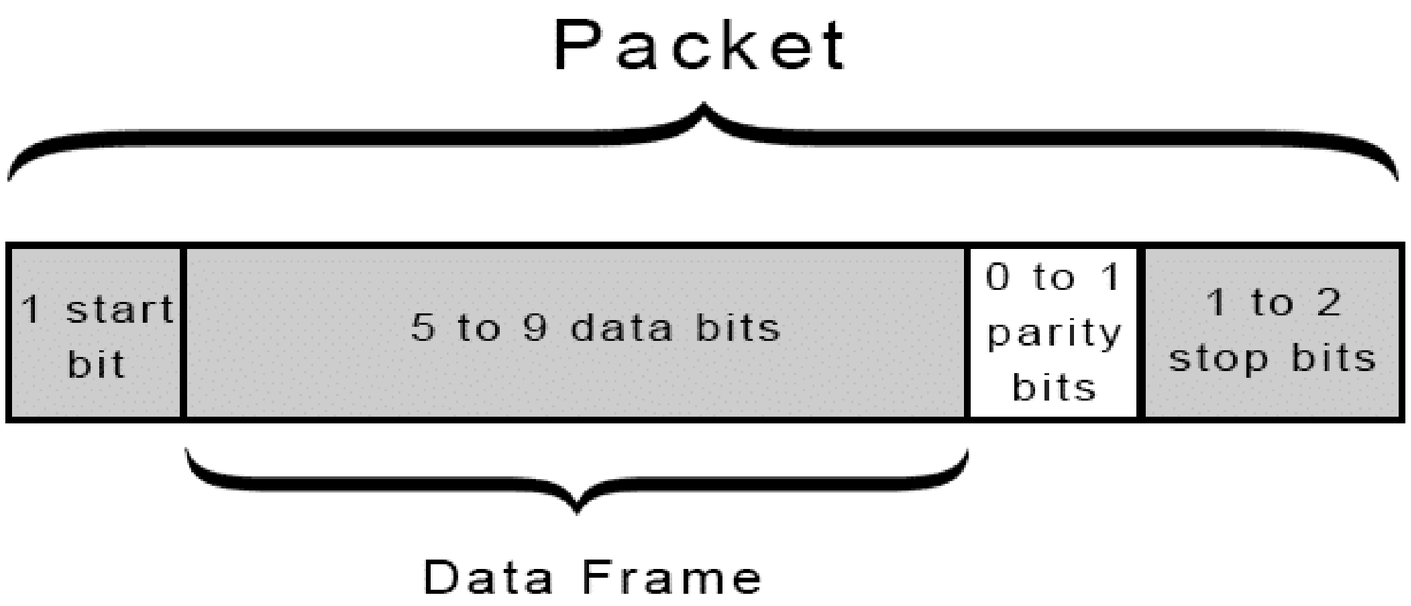


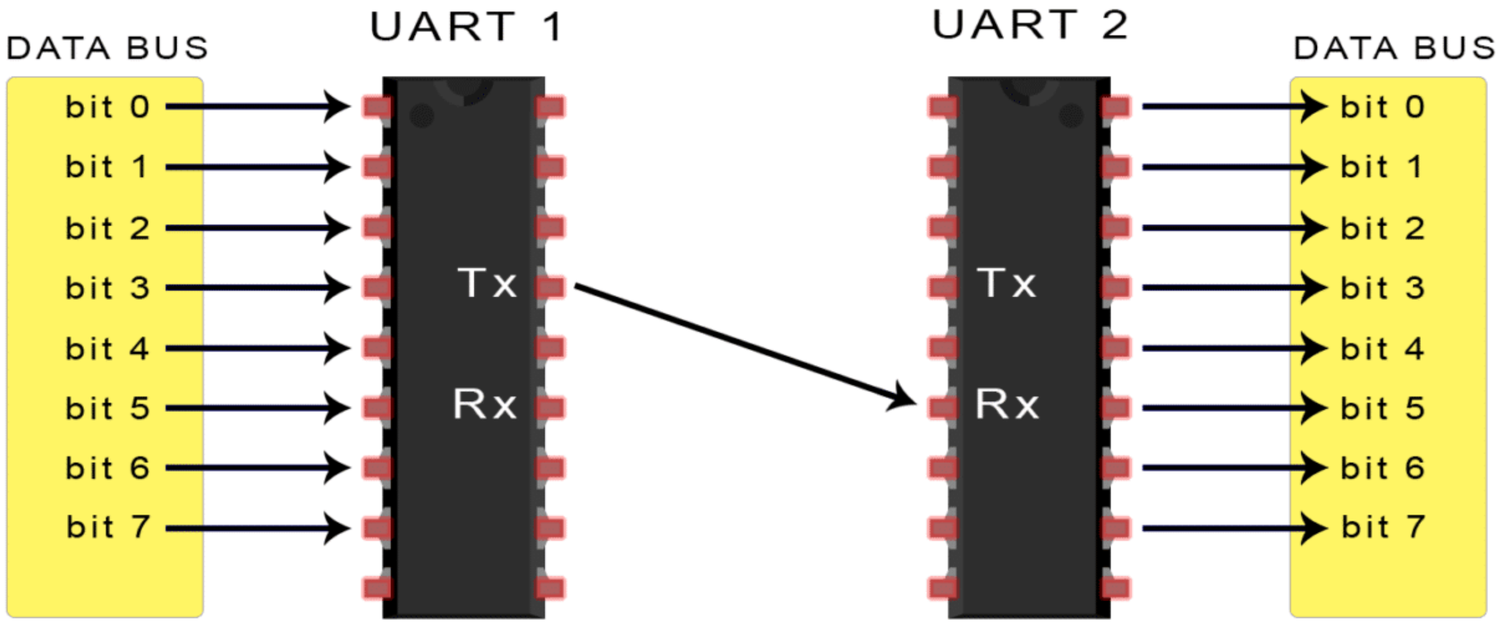
How UART Works

Data is transferred from the data bus to the transmitting UART in parallel form. After the transmitting UART gets the parallel data from the data bus, it adds a start bit, a parity bit, and a stop bit, creating the data packet.

Next, the data packet is output serially, bit by bit at the Tx pin. The receiving UART reads the data packet bit by bit at its Rx pin.

The receiving UART then converts the data back into parallel form and removes the start bit, parity bit, and stop bits. Finally, the receiving UART transfers the data packet in parallel to the data bus on the receiving end:


Packet
1 start bit
5 to 9 data bits
0 to 1 parity bits
1 to 2 stop bits
Data Frame


DATA BUS
bit 0
bit 1
bit 2
bit 3
bit 4
bit 5
bit 6
bit 7
UART 1
Tx
Rx
UART 2
Tx
Rx
DATA BUS
bit 0
bit 1
bit 2
bit 3
bit 4
bit 5
bit 6
bit 7

I2C: Inter-Integrated Circuits

Overview:
I2C is a serial communication protocol, so data is transferred bit by bit along a single wire (the SDA line).

Like SPI, I2C is synchronous, so the output of bits is synchronized to the sampling of bits by a clock signal shared between the master and the slave. The clock signal is always controlled by the master.

How It Works:
Master-Slave Protocol: I2C uses a master-slave architecture where the master (usually the microcontroller) controls communication with multiple slave devices.

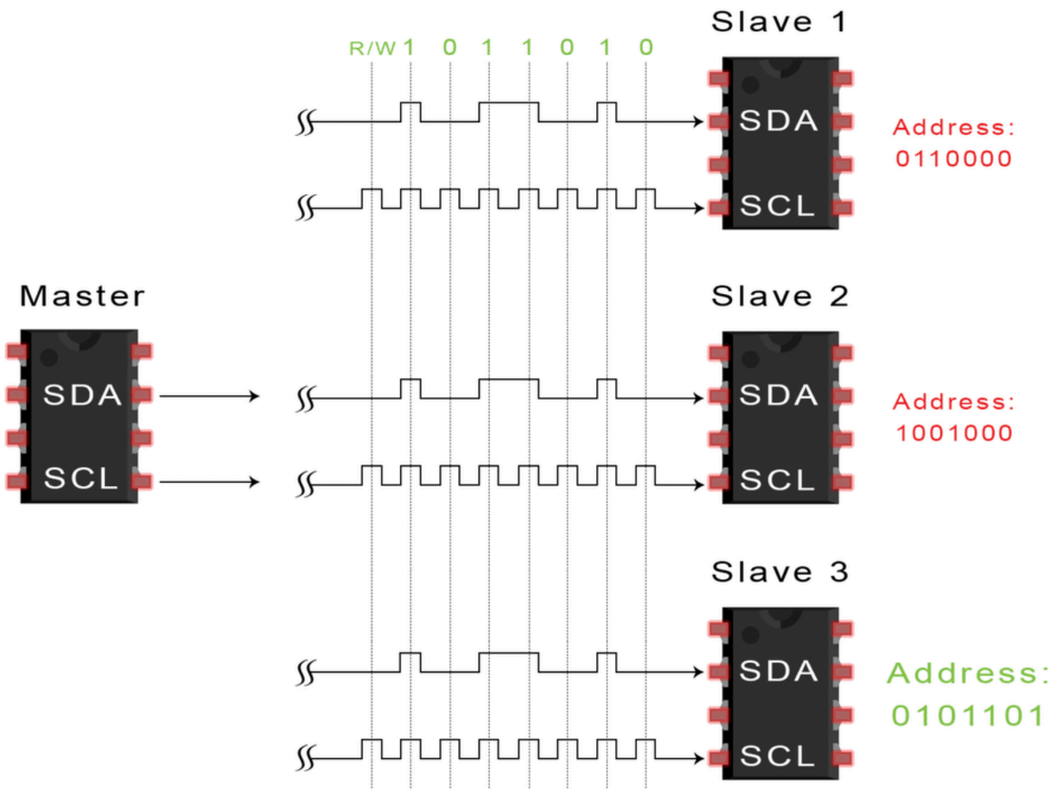
Addressing:
Each I2C device has a unique address, so multiple devices can share the same data and clock lines.

SDA (Serial Data) – The line for the master and slave to send and receive data.

SCL (Serial Clock) – The line that carries the clock signal.

I2C is a **serial communication protocol**, so data is transferred bit by bit along a single wire (the SDA line).

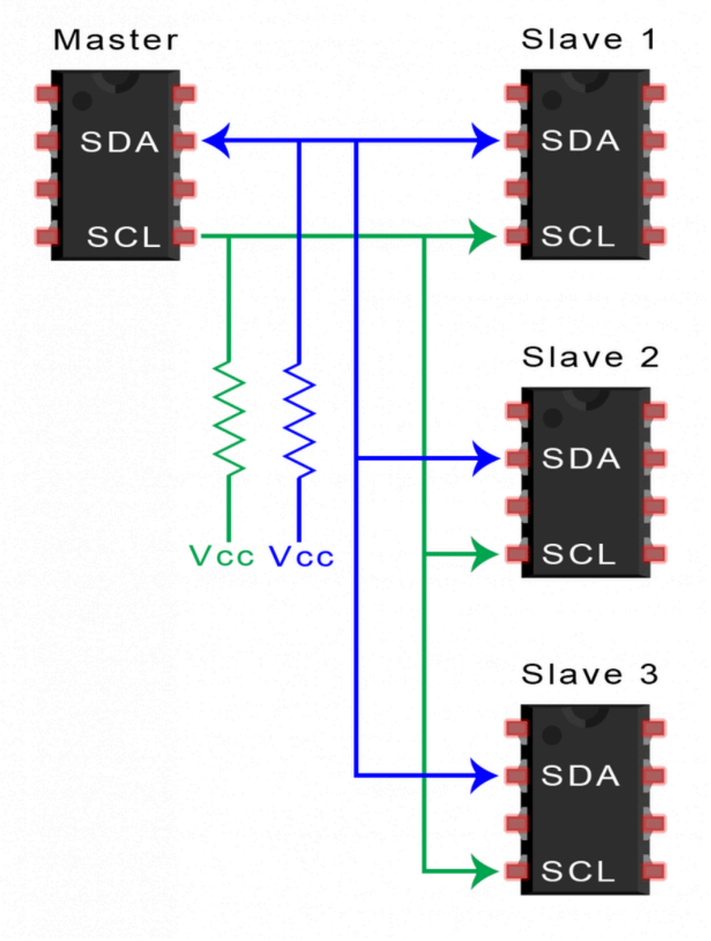
Like SPI, I2C is **synchronous**, so the output of bits is synchronized to the sampling of bits by a clock signal shared between the master and the slave.



Single Master with Multiple Slaves:
Because I2C uses addressing, multiple slaves can be controlled from a single master.

With a 7 bit address, **128** (2^7) unique address are available. Using 10 bit addresses is uncommon, but provides **1,024** (2^10) unique addresses.

To connect multiple slaves to a single master, wire them like this, with 4.7K Ohm pull-up resistors connecting the SDA and SCL lines to Vcc:



How I2C Works

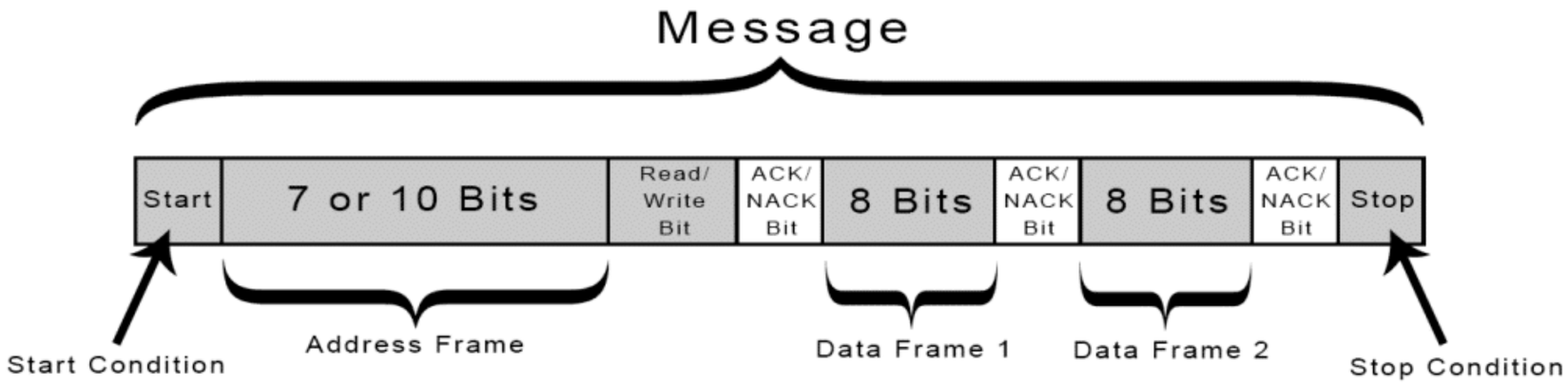
Start Condition: The SDA line switches from a high voltage level to a low voltage level before the SCL line switches from high to low.

Stop Condition: The SDA line switches from a low voltage level to a high voltage level after the SCL line switches from low to high.

Address Frame: A 7 or 10 bit sequence unique to each slave that identifies the slave when the master wants to talk to it.

Read/Write Bit: A single bit specifying whether the master is sending data to the slave (low voltage level) or requesting data from it (high voltage level).

ACK/NACK Bit: Each frame in a message is followed by an acknowledge/no-acknowledge bit. If an address frame or data frame was successfully received, an ACK bit is returned to the sender from the receiving device.



Addressing:
I2C doesn't have slave select lines like SPI, so it needs another way to let the slave know that data is being sent to it, and not another slave. It does this by addressing. The address frame is always the first frame after the start bit in a new message.

The Data Frame:
After the master detects the ACK bit from the slave, the first data frame is ready to be sent. The data frame is always 8 bits long, and sent with the most significant bit first.

WI-FI

How It Works:

Client and Server Roles: Wi-Fi can be used for connecting to external networks as a client or hosting a network as a server.

Protocols: Wi-Fi communication with CircuitPython often involves HTTP or MQTT for IoT applications, providing standard ways to send and receive data over the internet.

Protocol	Frequency	Maximum data rate
Legacy 802.11	2.4 GHz	2 Mbps
802.11b	2.4 GHz	11 Mbps
802.11g	2.4 GHz	54 Mbps
802.11n	2.4 GHz or 5 GHz	600 Mbps (theoretical)
802.11a	5 GHz	54 Mbps
802.11ac	5 GHz	1.3 Gbps